Ngingane Nhlume Lihle Exercise 02: SQL Fundamentals

SQL Aggregate Functions and Operators

01) The students Table

students

student_id	name	age	department
1	Alice	20	IT
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

1.
```
SELECT ~~department~~ DISTINCT department
FROM students ;
```

department
IT
HR
Finance

2.
```
SELECT department,
       AVG ( age ) AS avg-age
FROM studends
GROUP BY department;
```

department	avg-age
IT	20.5
HR	22
Finance	23

3. SELECT department,
COUNT (student_id) AS student_count
FROM students
GROUP BY Department
HAVING student_count >1 ;

department	student_count
IT	2
HR	2

4. SELECT *
FROM students
WHERE age BETWEEN 21 AND 23 ;

student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

5. SELECT *
FROM students
WHERE department = 'IT' AND age > 21
OR department = 'HR' AND age > 21 ;

student_id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

Q2) The courses Table

courses

course_id	course_name	department	credits
101	SQL Basics	IT	3
102	Python	IT	4
103	Data Science	IT	4
104	Excel	Finance	2
105	Statistics	HR	3

6.
```
SELECT department,
       SUM(credits) AS total_credits
FROM courses
GROUP BY department
HAVING total_credits > 5;
```

department	total_credits
IT	11

7.
```
SELECT *
FROM courses
WHERE credits <> 4;
```

course_id	course_name	department	credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

8. SELECT course_id,
course_name,
credits
FROM courses
ORDER BY credits DESC;

course_id	course_name	credits
102	Python	4
103	Data Science	4
101	SQL Basics	3

03). The enrollments Table

enrollments

enrollment_id	student_id	course_id	grade
1	1	101	85
2	2	102	78
3	3	103	90
4	4	104	88
5	5	105	82

9. SELECT
MAX(grade) AS max_grade,
MIN(grade) AS min_grade,
AVG(grade) AS avg_grade
FROM enrollments;

max_grade	min_grade	avg_grade
90	78	84.6

10.

```
SELECT course_id,
       COUNT(course_id) AS enrollment_count
FROM enrollments
~~Group~~ GROUP BY course_id;
```

course_id	enrollment_count
101	1
102	1
103	1
104	1
105	1

c) <u>The salaries Table</u>

salaries

employee_id	name	department	salary	bonus
1	Tom	IT	60 000	5000
2	Jerry	HR	55 000	4000
3	Spike	Finance	70 000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54 000	3500

11.

```
SELECT department,
       SUM(salary) AS total_salary,
       SUM(bonus) AS total_bonus
FROM salaries
GROUP BY department;
```

department	total_salary	total_bonus
IT	122 000	10 500
HR	109 000	7 500
Finance	70 000	6000

12. SELECT department,
AVG (salary) AS avg_salary
FROM salaries
GROUP BY department
HAVING avg_salary > 55 000;

department	avg_salary
IT	61 000
Finance	70 000

13. SELECT employee_id,
name,
salary,
bonus,
~~SUM (salary + bonu~~
SUM (salary + bonus) AS total_compensation
FROM salaries
GROUP BY employee_id,
name,
salary,
bonus
HAVING total_compensation > 60 000;

employee_id	name	salary	bonus	total_compensation
1	Tom	~~IT~~ 60 000	5000	65 000
3	Spike	~~Finance~~ 70000	6000	76 000
4	Tyke	~~IT~~ 62 000	5 500	67 500

05) The projects Table

projects

project_id	project_name	department	budget
1	AI App	IT	120 000
2	Payroll System	Finance	80 000
3	Dashboard	IT	150 000
4	Website	Marketing	60 000
5	HR Portal	HR	50 000

14.
```
SELECT department,
       SUM(budget) AS total_budget,
       AVG(budget) AS avg_budget
FROM projects
GROUP BY department
HAVING avg_budget > 70 000
```

department	total_budget	avg_budget
IT	170 000	135 000
Finance	80 000	80 000

15.
```
SELECT project_id, project_name, department, budget
-- For big datasets use SELECT *
FROM projects
WHERE budget BETWEEN 50 000 AND 120 000
AND department NOT IN ('Marketing');
```

project_id	project_name	department	budget
1	AI App	IT	120 000
2	Payroll System	Finance	80 000
~~3~~ 8			
5	HR Portal	HR	50 000